

194CS - PA Road Network Analysis

Brandon Wong II,

2026-03-05

Introduction

The roadNet-PA dataset found on Stanford's Network Analysis Project, or SNAP for short, is the subject of our analysis. This dataset represents the Pennsylvania road infrastructure as an undirected graph network where **nodes** (or vertices) are intersections or endpoints (dead-ends / cul-de-sacs) and **edges** are the roads that connect them. The network has rough 1.9 million nodes and 1.5 million edges which makes it a relatively comprehensive model of a real-world transportation system.

Our analysis has two primary objectives:

- “Town” Detection: Can we find distinct Functional Neighborhoods or “Towns” within major hubs using community detection algorithms?
- “Town” Connectivity & Resilience: How does the removal of high-betweenness articulation points impact the structural resilience of these “Towns”? In other words, is Pennsylvania's road infrastructure susceptible to major intersection closures or does it offer alternate paths for traversing the network?

Methodology

Data Processing

We imported the data using `read.table` and then called `graph_from_data_frame` to convert our dataset into an actual graph object. Upon running this, we noticed that we have a graph object with 1088092 nodes and 3083796 edges, which is exactly double the number of edges we should have in our network. What happened here, is that our .txt file represents an undirected edge by listing it twice, i.e A -> B and B -> A, meaning when we call our `graph_from_data_frame` function, it treats these as two separate edges between the two nodes and thus we get double the edges we should. To fix this, we called the `simplify` function on our imported network, which removes duplicate edges and self-loops, leaving us with the correct number of nodes and edges in our network.

Subgraph Generation

Due to how computationally expensive it is to visualize and analyze 1 million+ nodes, we needed to come up with a way to generate subgraphs. The sampling method we employed involves iteratively building out a neighborhood at an order of 15 steps from a high-degree node and then removing these used nodes for the next iteration. The idea behind our subgraph generation method is to mimic selecting urban centers and their surrounding areas. By iteratively choosing the highest-degree node of the network as the start node we build out neighborhoods which effectively gives us subgraphs representing 5 different urban centers of PA's road network.

Community Detection Algorithms

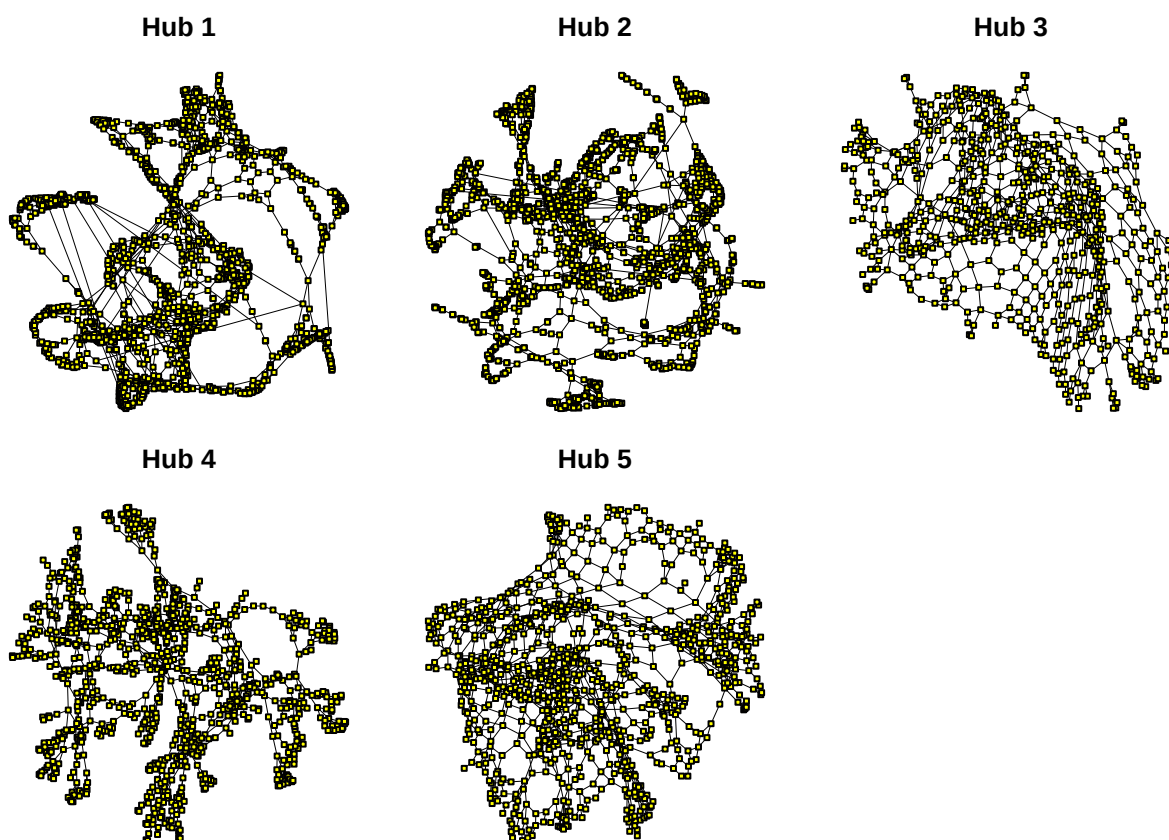
Walktrap and **Fast Greedy** are implemented on all 5 subnetworks to detect neighborhoods/blocks within each hub. Since they cluster the intersections with different mechanisms, we want to compare the performance of different algorithms and find the one that best fit our network with higher modularity.

The **Walktrap** uses the idea of random walks to identify densely connected “towns” within a network. The intuition is that if you were to take a short random walk from a node (3-5 steps), then you are more likely to stay within the same “town” then jump to a different one making this algorithm better at detecting smaller “towns” within a hub.

The **Fast Greedy** algorithm tries to maximize Modularity. It starts by treating each intersection as its own separate community, then greedily merges other communities that yield the largest increase in modularity. Making this algorithm good for looking at the global structure of the network and computationally quick. The downside to this algorithm is that it tends to miss smaller neighborhoods since it focuses on maximizing modularity.

Analysis and Results

Description and Visualization of Subgraphs



Above we can see the generated subgraphs. These subgraphs offer us a look into the topological structure of Pennsylvania’s Road Network. Since these were built-out using the `ego()` function and high-degree nodes as the origin, each of these subgraphs serve to represent a major region in Pennsylvania.

Hubs 1 and 2 we can see long-sweeping edges that bypass local clusters of intersections. This suggests that our high-degree nodes used to generate these subgraphs are located near major highways, where these

highways connect the local intersection clusters. Hubs 3 and 5 are characterized by their dense, grid-like structure. These are likely major metropolitan areas of Pennsylvania where the grid-like structure represents the urban planning, where there is multiple routes existing between 2 points. Hub 4 is the most unique out of the 5 subnetworks. We can see a clear central core with “arms” that branch out from it. These “arms” are not connected to each other and only connected through the central core, meaning if you removed the node connecting the core to this arm, then this arm would be completely isolated from the network. This shows to us how central town or city acts as the connection to smaller towns or rural roads.

Overall, these hubs reveal various different topological structures of Pennsylvanis Road Network. Revealing why we chose to build out networks from high-degree nodes.

Subnetwork Metrics

Table 1: Hub Mectrics

Name	Hub 1	Hub 2	Hub 3	Hub 4	Hub 5
Nodes	1415	1680	666	989	945
Edges	2176	2254	1092	1251	1489
Avg_Degree	3.08	2.68	3.28	2.53	3.15
Transitivity	0.07	0.06	0.05	0.07	0.04
Density	0.00218	0.00160	0.00493	0.00256	0.00334
CC	0.07	0.06	0.05	0.06	0.05
BC	0.40	0.30	0.19	0.36	0.20
Diameter	30	30	30	30	30
Artic_Pts	134	381	32	287	62

The nodes and edges describe the size of the network, where nodes are intersections/endpoints and edges are the road segments connecting them. Our hubs range from 666 to ~1400 nodes where Hub 1 is our largest urban center with 1415 nodes.

The next metric we looked at is the **Average Degree**, which is the average number of edges connected to a node. However, unlike social networks where a person can have 500 friends (connections/edges), our road network is limited to 2d space, meaning the number of connections a node can have is restricted to the space it lies within. Due to this, the avg degree distribution from ~2.53 to ~3.28, makes a lot of sense. Hub 3 has an average degree of 3.28 which suggests more T-junctions or Crossroads, whereas an average degree of 2.53 for Hub 4 suggests that there are more dead-ends or long-stretches with little intersections.

Another metric we looked at **Transitivity** which is the probability that neighbors of a certain node are also connected to each other forming small triangles. The values we see in our road networks are really low, ranging from 0.05-0.07. While in social networks, transitivity is usually very high, for road networks, low transitivity actually indicates how efficient the road network is. Road networks operate on a 2d plane, unlike Social Networks multi-dimensional domain, meaning that if road networks were constructed with diagonal shortcuts connecting two node endpoints, then this would be incredibly expensive and hard to do in practice. As such, efficient road networks are generally organized in squares/rectangles, meaning that a standard city grid will have a transitivity of 0, despite being highly redundant and easy to traverse. With this in mind, Hub 5 has a transitivity value of 0.04, which suggests that this road network is a highly structured grid connected through square blocks rather than triangles. Hub 4 has the highest transitivity of 0.07 which suggests a more freeform road structure, which could be due to certain geographical features forcing roads to merge and indirectly creating “triangles” that our transitivity function picks up.

We also looked at **Density** which is the proportion of actual edges in the network to the total possible number of edges in the network if every node was connected to every other node. The density values we see range from ~0.00160 to ~0.00493, which are extremely low. This is to be expected, because in a road network, we cannot connect each intersection to every other intersection because roads cannot overlap infinitely. As

such, the extremely low values we see here is expected due to physical restrictions of road networks. It is to be noted that Hub 3 has the highest density of 0.00493 which suggests this is the most compact and interconnected urban center.

CC or **Closeness Centrality** measures how close a node is to all other nodes in the network based on the shortest path distance. However, what we use is the **Closeness Centralization** which measures how much a single node is closer to other nodes than the average node. Hubs with high CC are generally very compact, having a clear central region from which you can reach any other region with few turns. The CC values we see across all hubs are very small, ranging from 0.05-0.07 implying that Pennsylvania’s road hubs are decentralized, there isn’t one intersection that allows access to all other intersections. This makes sense as states generally don’t just have one centralized town that connects all others, but instead is a mix of smaller and larger towns that distributes the closeness measure across these local centers.

BC or **Betweenness Centrality** counts the number of times a node acts as a “bridge” along the shortest path between two other nodes whereas **Betweenness Centralization** measures how much the networks traffic flow is bottlenecked through a small number of nodes. This metric is crucial for our later resilience analysis, where if a High BC node is closed, it does not just affect traffic flow locally, but also globally (in the context of the hub), forcing traffic to use a longer, less efficient route. Hub 1 has the highest BC score of 0.4 which means to traverse across this network, traffic must go through a few specific intersections, making it vulnerable to single points of failures. Hub 3 has the lowest BC value of 0.19, which implies this Hub has a grid-like structure, where no single intersection controls the flow of traffic and instead offers various paths to get from point A to point B, making it more resilient to individual intersection closures.

Diameter is the “longest-shortest path” between any two nodes in the network. This metric isn’t all that interesting for interpretation as our diameter values are all 30, which is simply the result of generating our hubs using the ego() function at an order of 15 steps.

Artic_Pts or **Articulation Points** are the nodes where, if removed, would break the network into two or more disconnected pieces. Hub 2 has by far the most number of articulation points with 381, making it highly susceptible to road closures. Hub 3 has the least with 32, meaning that this Hub is much more robust than the rest, where if one intersection closes, a person can just use an alternate route to go around.

Community Detection Results and Analysis

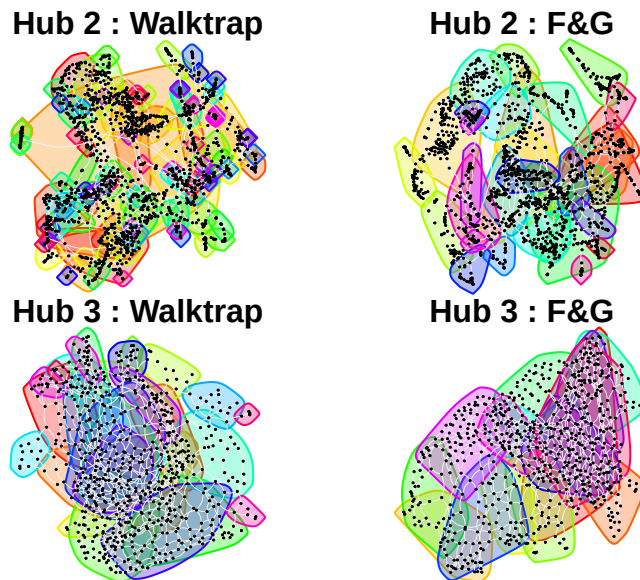


Table 2: Community Detection Results

Name	Hub 1	Hub 2	Hub 3	Hub 4	Hub 5
Modularity_Walk	0.87	0.84	0.81	0.86	0.85
Modularity_FG	0.89	0.90	0.82	0.90	0.85
Walk_Comms	45	112	27	67	37
FG_Comms	26	31	16	24	19

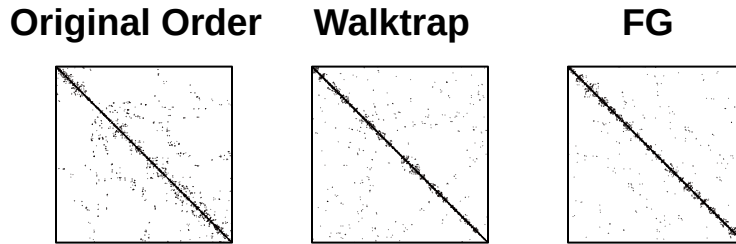
Modularity measures how well partitions divide the network into communities. In general, a modularity score of greater than 0.3 indicates a meaningful community structure. Looking at our results, we can see that all hubs show extremely high modularity, ranging from **0.81 to 0.87** for the Walktrap algorithm and **0.82 to 0.90** for the Fast and Greedy algorithm.

Fast Greedy consistently finds fewer communities with a range of 16 communities in Hub 3 to 31 in Hub 2 whereas the Walktrap finds much more ranging from 27 communities in Hub 3 to 112 in Hub 2. The reason for this difference in number of communities detected is due to how each algorithm works. The Walktrap simulates random walks and due to this, it is good at identifying smaller communities, leading it to detect more communities than the Fast and Greedy approach which merges the nodes that result in the highest increase in modularity, leading it to detect less communities. Despite this disconnect, both algorithms agree on our Hubs having strong community structure.

Walktrap produces more smaller and fragmented communities, which is visible from the small isolated colored clusters near the red boundaries, reflecting its sensitivity to local structure through short random walks. The small communities detected are likely to be neighborhoods or blocks that are closely connected by local streets rather than major roadways connecting different part of the town.

In contrast, Fast Greedy produces fewer, larger, and more cohesive communities, which is consistent with its global modularity optimization that tends to miss small communities and merge those communities into larger ones. The larger communities detected are more widespread than those in walktrap and thus likely to be larger districts or regions of the town/city that includes multiple neighborhoods or blocks, and these communities are likely connected by major roadway in the town with high betweenness centrality.

Adjacency Matrix Reordering



As we can see above, the adjacency matrix with original node order appears to have a diagonal pattern already, and this is due to how we subsample our graph using `neighborhood()`. All the intersections are collected sequentially as the neighbors to the starting node, and thus all neighboring intersections tend to have similar indexes, causing those to cluster along the diagonal. However, there are a considerable amount of connections spreading away the diagonal. After reordering, the diagonal trend appears as expected but there are very few sparse connections outside of the diagonal, indicating that the community detection algorithms tightly cluster intersections within the same community.

Resilience Analysis and Results

We simulate targeted scenarios such as road closures, infrastructure failures, or disaster damage at intersections with the most traffic flow in the road network. The goal is to observe how the town’s connectivity and structure degrade under such disruptions, specifically examining whether the town fragments into isolated regions, are there any unreachable intersections, and how overall connectivity metrics change after each removal. This provides insight into the town’s infrastructural vulnerability and helps identify which intersections are most critical to maintain for ensuring a functional road network.

Removing high betweenness articulation intersection is impactful because these intersection serve as connectors whose removal disconnects the network and as major traffic bottlenecks through which the most shortest paths flow. Their failure may cause fragmentation and disruption to the reachability of road network.

Name	Components	Disconnected_Pairs	Diameter	BC	CC	Modularity	Communities
Hub 1	1	0	30	0.4004	0.0687	0.8706	45
Removal Hub 1	3	2817	32	0.3992	NaN	0.8717	39
Hub 2	1	0	30	0.2988	0.0650	0.8409	112
Removal Hub 2	3	3347	31	0.2984	NaN	0.8456	70
Hub 3	1	0	30	0.1884	0.0500	0.8052	27
Removal Hub 3	1	0	31	0.2047	0.0444	0.8152	24
Hub 4	1	0	30	0.3573	0.0610	0.8641	67
Removal Hub 4	2	983	30	0.3579	NaN	0.8638	74
Hub 5	1	0	30	0.1956	0.0532	0.8461	37
Removal Hub 5	1	0	33	0.1993	0.0531	0.8300	46

Choosing optimal number of removed High-Betweenness Articulation Points

Selecting appropriate number of intersection removals is also important, because we want a reasonable numbers to simulate the real-world scenarios of road closures and significant enough to observe the disruption on the traffic system. Removing one intersection might not stress the traffic system but removing ten are not as common in the real world that rarely this many intersections would be shut down simultaneously.

Mectrics Analysis

The removal has a measurable but relatively moderate impact on the traffic system within each hub. Connectivity between neighborhoods/districts is disrupted, so in hubs 1, 2, and 4, towns are divided into 2 or 3 components, with no connecting roads, leaving residents unable to access all parts of their towns due to the loss of major roadways. In addition, the number of disconnected intersection pairs rises significantly, meaning that residents can no longer commute freely from place to place, as well as the percentage of disconnected intersections within hub1, 2, and 4. The longest shortest path (diameter) increases slightly, and on average, people need to go through more intersections. Another notable feature is the closeness centrality of hubs 1, 2, and 4: they are NA, indicating isolated intersections in the network with zero connecting paths, as closeness centrality cannot be calculated when the network is fragmented. Transitivity remains stable as those disconnected intersections are not taken into account.

In terms of community structure, after removal, modularity for all hubs remains stable, but the algorithm struggles to detect communities for hubs 1, 2, and 3, where fewer neighborhoods are recognized. Rather, multiple neighborhoods/blocks are probably merged into larger communities. This might be because those 5 major intersections serve as boundaries between neighborhoods and/or communities, and shutting them down would blur those boundaries, leading the algorithm to consolidate nearby communities into a single large community rather than separate neighborhoods/blocks.

Overall, all 5 hubs demonstrate good resilience when major intersections are shut down. However, hubs 1, 2, and 4 are more vulnerable, as removing the 5 intersections with the most traffic would segment the town into segregated regions. The fragmentation would lead to serious real-world concerns if an emergency situation occurred: residents and emergency vehicles could not reach other parts of the town without an alternative route. The traffic system of Hubs 1, 2, and 4 would experience a relatively significant interruption, as connectivity and reachability are both negatively impacted. Commute costs are likely increasing as people may need to choose another route with a longer commute time and more gas.

Conclusion

Our analysis has shown that Pennsylvania’s road networks are characterized as having low transitivity, low density, and an average degree of roughly 3 across all hubs. These metrics reveal the underlying structure of Pennsylvania’s road structure, where low transitivity indicates the grid-like structure of road networks, and low density and average degree of 3 highlights the physical limitations of road networks.

Through our community detection results, we found that we have high modularity across all five subgraphs, which confirms that we can detect functional “towns” within our subnetworks. Additionally, our resilience analysis demonstrated that while all Hubs 3 and 5 demonstrate good resilience to major intersection shut-downs, Hubs 1, 2, and 4 are much more vulnerable to the shut-down of major intersections, where the removal of these nodes/intersections results in the fragmentation of the Hub, leading to an inability to efficiently traverse the network.

Since roadNet-PA contains no edge or node attributes beyond a basic name attribute for the nodes, future analysis would benefit from merging external datasets to add edge weights such as road length, speed limits, traffic reports (accidents/closures), road type (highway vs local street), or the intersection type (stoplight/stop sign).

References

“SNAP: Network Datasets: Pennsylvania Road Network.” Stanford.edu, 2026, snap.stanford.edu/data/roadNet-PA.html. Accessed 6 Mar. 2026.

Appendix 1

Code Used In Report

```
knitr::opts_chunk$set(echo = TRUE, warning = FALSE)
library(igraph)
library(Matrix)
library(dplyr)
library(kableExtra)
library(gridExtra)
library(lattice)
```

```

# Margin helpers
def_marg <- c(5.1, 4.1, 4.1, 2.1)
no_marg  <- c(0, 0, 0, 0)

pa_road_edges <- read.table(
  "roadNet-PA.txt",
  comment.char = "#",
  col.names = c("from", "to")
)

pa_road_network <- graph_from_data_frame(pa_road_edges, directed=FALSE)

# Removes duplicate edges / self-loops
simple_pa_road <- igraph::simplify(pa_road_network)
set.seed(2249282)
current_network <- simple_pa_road
subgraph_list_hub <- list()
num_subgraphs <- 5

par(mfrow = c(2, 3), mar = c(0, 1, 1, 1))

for (i in 1:num_subgraphs) {

  # Calculate degrees and find the top node
  deg <- degree(current_network, mode = "all")
  sorted_deg <- sort(deg, decreasing = TRUE, index.return = TRUE)
  start_node <- sorted_deg$ix[1]

  # Extract the neighborhood (order 15)
  nodes_to_extract <- ego(current_network, order = 15, nodes = start_node, mode = "all")[[1]]

  # Create the subgraph
  sub <- induced_subgraph(current_network, nodes_to_extract)

  # Store it in our list in case we want to analyze it later
  subgraph_list_hub[[i]] <- sub

  # Plot the subgraph immediately
  plot(sub,
    layout = layout_nicely,
    main = paste("Hub", i),
    vertex.color = "yellow",
    vertex.frame.color = "black",
    vertex.shape = "square",
    vertex.size = 3,
    vertex.label = NA,
    edge.color = "black",
    edge.lty = 1,
    edge.width = 0.5,
    edge.arrow.size = 0.2)

  # Update the network by removing the extracted nodes for the next iteration
  current_network <- delete_vertices(current_network, nodes_to_extract)
}

```



```

}
results_hub <- data.frame()
sub_names <- c("Hub 1", "Hub 2", "Hub 3", "Hub 4", "Hub 5")
for(i in 1:length(subgraph_list_hub)){
  g <- subgraph_list_hub[[i]]
  g <- simplify(g)

  # Build row
  row <- data.frame(
    Name = sub_names[i],
    Nodes = round(vcount(g), 2),
    Edges = round(ecount(g), 2),
    Avg_Degree = round(mean(degree(g)), 2),
    Transitivity = round(transitivity(g), 2),
    Density = round(edge_density(g), 5),
    CC = round(centr_clo(g, mode="all", normalized=T)$centralization, 2),
    BC = round(centr_betw(g, directed=F, normalized=T)$centralization, 2),
    Diameter = round(diameter(g), 2),
    Artic_Pts = length(articulation_points(g))
  )
  results_hub <- rbind(results_hub, row)
}

kable(t(results_hub), caption = "Hub Metrics")
all_colors <- colors()
vibrant_colors <- all_colors[!grepl("gray|grey", all_colors)]
unique_palette <- sample(vibrant_colors)

par(mfrow = c(2,2), mar = c(0, 1, 1, 0))
# Loop through each subgraph in your list
for(i in c(2,3)) {
  g <- subgraph_list_hub[[i]]

  cw <- cluster_walktrap(g)
  cfg <- cluster_fast_greedy(g)

  # Walktrap
  num_comms_cw <- length(unique(membership(cw)))
  v_colors_cw <- unique_palette[1:num_comms_cw]

  plot(cw, g,
    layout = layout_nicely,
    vertex.size = 1,
    vertex.label = NA,
    vertex.color = v_colors_cw[membership(cw)],
    edge.color = "grey95",
    edge.width = 0.5,
    edge.curved = 0.3,
    main = paste("Hub", i, ": Walktrap"))

  # Fast & Greedy
  num_comms_cfg <- length(unique(membership(cfg)))
  v_colors_cfg <- unique_palette[1:num_comms_cfg]

```

```

plot(cfg, g,
      layout = layout_nicely,
      vertex.size = 1,
      vertex.label = NA,
      vertex.color = v_colors_cfg[membership(cfg)],
      edge.color = "grey95",
      edge.width = 0.5,
      edge.curved = 0.3,
      main = paste("Hub", i, ": F&G"))
}

results_com <- data.frame()
sub_names <- c("Hub 1", "Hub 2", "Hub 3", "Hub 4", "Hub 5")
for(i in 1:length(subgraph_list_hub)){
  g <- subgraph_list_hub[[i]]
  g <- simplify(g)

  # Community Detection
  cw <- cluster_walktrap(g)
  cfg <- cluster_fast_greedy(g)

  # Build row
  row <- data.frame(
    Name = sub_names[i],
    Modularity_Walk = round(modularity(cw), 2),
    Modularity_FG = round(modularity(cfg), 2),
    Walk_Comms = round(length(unique(membership(cw))), 2),
    FG_Comms = round(length(unique(membership(cfg))), 2)
  )
  results_com <- rbind(results_com, row)
}

kable(t(results_com),caption = "Community Detection Results")
cw <- cluster_walktrap(subgraph_list_hub[[1]])
cfg <- cluster_fast_greedy(subgraph_list_hub[[1]])

# Original (unordered) adjacency matrix
A <- as_adjacency_matrix(subgraph_list_hub[[1]], sparse = TRUE)
p1 <- image(A, main = "Original Order", col.region = c("red"), cex = 0.8, scales = list(draw = FALSE),
            xlab = "", ylab = "", sub = FALSE)

# Reorder by Walktrap community membership
ord <- order(cw$membership)
A_reordered <- A[ord, ord]

p2 <- image(A_reordered,
            main = "Walktrap", col.region = c("red"), cex = 0.8, scales = list(draw = FALSE),
            xlab = "", ylab = "", sub = FALSE)

# Reorder by FG
ord1 <- order(cfg$membership)

```

```

A_reordered1 <- A[ord1, ord1]

p3 <- image(A_reordered1,
  main = "FG", col.region = c("red"), cex = 0.8, scales = list(draw = FALSE),
  xlab = "", ylab = "", sub = FALSE)

all_plots <- list(p1, p2, p3)

do.call(grid.arrange, c(all_plots, ncol = 3))
sub1 <- subgraph_list_hub[[1]]
sub2 <- subgraph_list_hub[[2]]
sub3 <- subgraph_list_hub[[3]]
sub4 <- subgraph_list_hub[[4]]
sub5 <- subgraph_list_hub[[5]]

artic_points1 <- articulation_points(sub1)
# cat("Number of articulation points for hub 1:", length(artic_points1), '\n')
betweenness_sub1 <- betweenness(sub1, v = artic_points1, directed = FALSE)
high_between_artic_node1 <- sort(betweenness_sub1, decreasing = T, index.return = T)$ix[1:5]
artic_points2 <- articulation_points(sub2)
# cat("Number of articulation points for hub 2:", length(artic_points2), '\n')
betweenness_sub2 <- betweenness(sub2, v = artic_points2, directed = FALSE)
high_between_artic_node2 <- sort(betweenness_sub2, decreasing = T, index.return = T)$ix[1:5]
artic_points3 <- articulation_points(sub3)
# cat("Number of articulation points for hub 3:", length(artic_points3), '\n')
betweenness_sub3 <- betweenness(sub3, v = artic_points3, directed = FALSE)
high_between_artic_node3 <- sort(betweenness_sub3, decreasing = T, index.return = T)$ix[1:5]
artic_points4 <- articulation_points(sub4)
# cat("Number of articulation points for hub 4:", length(artic_points4), '\n')
betweenness_sub4 <- betweenness(sub4, v = artic_points4, directed = FALSE)
high_between_artic_node4 <- sort(betweenness_sub4, decreasing = T, index.return = T)$ix[1:5]
artic_points5 <- articulation_points(sub5)
# cat("Number of articulation points for hub 5:", length(artic_points5), '\n')
betweenness_sub5 <- betweenness(sub5, v = artic_points5, directed = FALSE)
high_between_artic_node5 <- sort(betweenness_sub5, decreasing = T, index.return = T)$ix[1:5]
# Removing High-Betweenness Articulation Points
removal_sub1 <- delete_vertices(sub1, high_between_artic_node1)
removal_sub2 <- delete_vertices(sub2, high_between_artic_node1)
removal_sub3 <- delete_vertices(sub3, high_between_artic_node1)
removal_sub4 <- delete_vertices(sub4, high_between_artic_node1)
removal_sub5 <- delete_vertices(sub5, high_between_artic_node1)

removal_results <- data.frame()
removal_subs <- list(sub1, removal_sub1, sub2, removal_sub2, sub3, removal_sub3, sub4, removal_sub4, sub5, removal_sub5)
sub_names <- c("Hub 1", "Removal Hub 1", "Hub 2", "Removal Hub 2", "Hub 3", "Removal Hub 3", "Hub 4", "Removal Hub 4", "Hub 5", "Removal Hub 5")

for(i in 1:length(removal_subs)){
  g <- removal_subs[[i]]

  # Community Detection
  cw <- cluster_walktrap(g)

  # Build row

```

```

row <- data.frame(
  Name = sub_names[i],
  Components = components(g)$no,
  Disconnected_Pairs = distance_table(g, directed = FALSE)$unconnected,
  Diameter = diameter(g),
  BC = round(centr_betw(g, directed = F)$centralization,4),
  CC = round(centr_clo(g)$centralization,4),
  Modularity = round(modularity(cw),4),
  Communities = length(unique(membership(cw)))
)
removal_results <- rbind(removal_results, row)
}

removal_results %>%
  kable() %>%
  kable_styling(bootstrap_options = "striped", font_size = 8) %>%
  row_spec(c(1,2), background = "#FFE4E1") %>% # Hub 1
  row_spec(c(3,4), background = "#E1F5E1") %>% # Hub 2
  row_spec(c(5,6), background = "#E1E8FF") %>% # Hub 3
  row_spec(c(7,8), background = "#FFF8E1") %>% # Hub 4
  row_spec(c(9,10), background = "#F3E1FF")
all_colors <- colors()
vibrant_colors <- all_colors[!grepl("gray|grey", all_colors)]
unique_palette <- sample(vibrant_colors)

par(mfrow = c(5,2), mar = c(0, 1, 1, 0))
# Loop through each subgraph in your list
for(i in 1:length(subgraph_list_hub)) {
  g <- subgraph_list_hub[[i]]

  cw <- cluster_walktrap(g)
  cfg <- cluster_fast_greedy(g)

  # Walktrap
  num_comms_cw <- length(unique(membership(cw)))
  v_colors_cw <- unique_palette[1:num_comms_cw]

  plot(cw, g,
    layout = layout_nicely,
    vertex.size = 1,
    vertex.label = NA,
    vertex.color = v_colors_cw[membership(cw)],
    edge.color = "grey95",
    edge.width = 0.5,
    edge.curved = 0.3,
    main = paste("Hub", i, ": Walktrap"))

  # Fast & Greedy
  num_comms_cfg <- length(unique(membership(cfg)))
  v_colors_cfg <- unique_palette[1:num_comms_cfg]

  plot(cfg, g,

```

```

    layout = layout_nicely,
    vertex.size = 1,
    vertex.label = NA,
    vertex.color = v_colors_cfg[membership(cfg)],
    edge.color = "grey95",
    edge.width = 0.5,
    edge.curved = 0.3,
    main = paste("Hub", i, ": F&G"))
}
all_plots <- list()

for(i in 1:length(subgraph_list_hub)) {
  g <- subgraph_list_hub[[i]]

  cw <- cluster_walktrap(g)
  cfg <- cluster_fast_greedy(g)

  # Original (unordered) adjacency matrix
  A <- as_adjacency_matrix(g, sparse = TRUE)

  # 1. Original Plot
  # We pass scales, xlab, and ylab directly to avoid the par.settings error
  p1 <- image(A,
    main = list(label = paste("Original: Hub", i), cex = 0.8),
    col.regions = "red",
    scales = list(draw = FALSE),
    xlab = NULL, ylab = NULL, sub = FALSE)

  # 2. Reorder by Walktrap
  ord1 <- order(cw$membership)
  A_reordered1 <- A[ord1, ord1]
  p2 <- image(A_reordered1,
    main = list(label = paste("Walktrap: Hub", i), cex = 0.8),
    col.regions = "red",
    scales = list(draw = FALSE),
    xlab = NULL, ylab = NULL, sub = FALSE)

  # 3. Reorder by FG
  ord2 <- order(cfg$membership)
  A_reordered2 <- A[ord2, ord2]
  p3 <- image(A_reordered2,
    main = list(label = paste("FG: Hub", i), cex = 0.8),
    col.regions = "red",
    scales = list(draw = FALSE),
    xlab = NULL, ylab = NULL, sub = FALSE)

  all_plots <- c(all_plots, list(p1, p2, p3))
}

# Arrange the final grid
do.call(grid.arrange, c(all_plots, ncol = 3))

```

Appendix 2

1. Tools and Packages

We used R and RStudio as our programming language and platform to perform our analysis.

The packages we used were:

- *igraph*: Used for network manipulation, community detection, and metric calculations.
- *Matrix*: Used to create the adjacency matrices.
- *dplyr*: Used for data manipulation and cleaning
- *kableExtra*: Used for creating better tables.
- *lattice*: Used to render Adjacency Matrices
- *gridExtra*: Used to plot Adjacency Matrices on a single page

2. Additional Code

2.1 All Community Detection Visualizations

```
all_colors <- colors()
vibrant_colors <- all_colors[!grepl("gray|grey", all_colors)]
unique_palette <- sample(vibrant_colors)

par(mfrow = c(5,2), mar = c(0, 1, 1, 0))
# Loop through each subgraph in your list
for(i in 1:length(subgraph_list_hub)) {
  g <- subgraph_list_hub[[i]]

  cw <- cluster_walktrap(g)
  cfg <- cluster_fast_greedy(g)

  # Walktrap
  num_comms_cw <- length(unique(membership(cw)))
  v_colors_cw <- unique_palette[1:num_comms_cw]

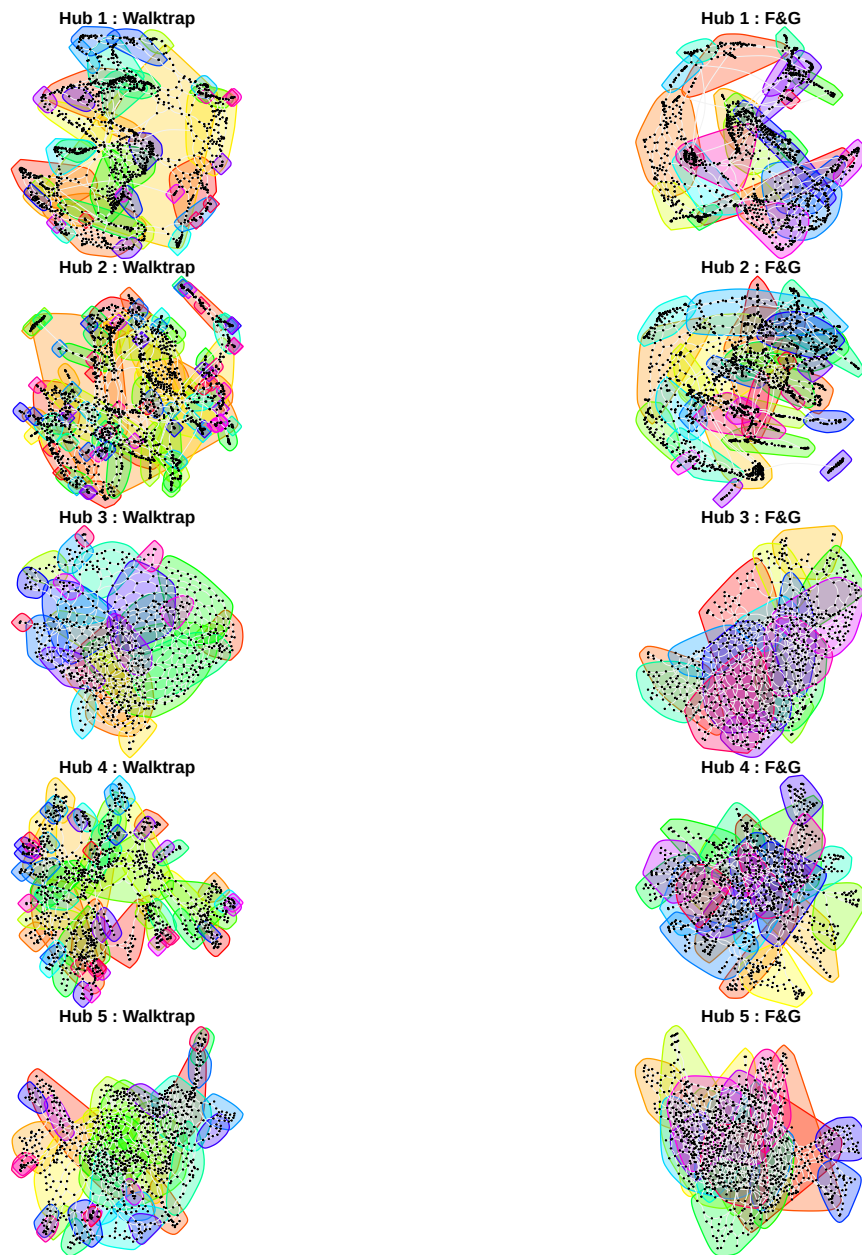
  plot(cw, g,
        layout = layout_nicely,
        vertex.size = 1,
        vertex.label = NA,
        vertex.color = v_colors_cw[membership(cw)],
        edge.color = "grey95",
        edge.width = 0.5,
        edge.curved = 0.3,
        main = paste("Hub", i, ": Walktrap"))

  # Fast & Greedy
  num_comms_cfg <- length(unique(membership(cfg)))
  v_colors_cfg <- unique_palette[1:num_comms_cfg]
```

```

plot(cfg, g,
      layout = layout_nicely,
      vertex.size = 1,
      vertex.label = NA,
      vertex.color = v_colors_cfg[membership(cfg)],
      edge.color = "grey95",
      edge.width = 0.5,
      edge.curved = 0.3,
      main = paste("Hub", i, ": F&G"))
}

```



2.2 All Adjacency Matrices (Original and Reordered)

```
all_plots <- list()

for(i in 1:length(subgraph_list_hub)) {
  g <- subgraph_list_hub[[i]]

  cw <- cluster_walktrap(g)
  cfg <- cluster_fast_greedy(g)

  # Original (unordered) adjacency matrix
  A <- as_adjacency_matrix(g, sparse = TRUE)

  # 1. Original Plot
  # We pass scales, xlab, and ylab directly to avoid the par.settings error
  p1 <- image(A,
    main = list(label = paste("Original: Hub", i), cex = 0.8),
    col.regions = "red",
    scales = list(draw = FALSE),
    xlab = NULL, ylab = NULL, sub = FALSE)

  # 2. Reorder by Walktrap
  ord1 <- order(cw$membership)
  A_reordered1 <- A[ord1, ord1]
  p2 <- image(A_reordered1,
    main = list(label = paste("Walktrap: Hub", i), cex = 0.8),
    col.regions = "red",
    scales = list(draw = FALSE),
    xlab = NULL, ylab = NULL, sub = FALSE)

  # 3. Reorder by FG
  ord2 <- order(cfg$membership)
  A_reordered2 <- A[ord2, ord2]
  p3 <- image(A_reordered2,
    main = list(label = paste("FG: Hub", i), cex = 0.8),
    col.regions = "red",
    scales = list(draw = FALSE),
    xlab = NULL, ylab = NULL, sub = FALSE)

  all_plots <- c(all_plots, list(p1, p2, p3))
}

# Arrange the final grid
do.call(grid.arrange, c(all_plots, ncol = 3))
```